

● METODOLOGIA SYSTEM UX

Como construir User Stories.

Pensar componente e tela **antes** de abrir o Figma. Um guia direto para transformar dor de pesquisa em backlog pronto para o Jira — sem corporativês, sem palpite, com o caminho real da necessidade até a interface.

● O QUE VEM

Conteúdo do **playbook.**

01	O que é uma User Story A menor unidade de valor · necessidade, não solução · a regra de ouro	p. 03
02	A estrutura base Como · Quero · Para que · certo vs. errado	p. 04
03	O card completo ID · título · prioridade · story points · épico · sprint	p. 05
04	Critérios de aceitação O coração do card · a ponte para os componentes	p. 06
05	Épicos Agrupar por objetivo · a cor que organiza o board	p. 07
06	Sprints Sequenciar dos fundamentos ao avançado	p. 08
07	O fluxo completo (System UX) Da discovery ao Figma · a ordem importa	p. 09
08	Mapa de componentes & telas O entregável-ponte entre história e interface	p. 10
09	Checklist de qualidade O que conferir antes de mandar para o backlog	p. 11
10	Modelo pronto Copie, preencha · exemplo completo	p. 12

● O CONCEITO

O que é uma User Story.

Uma User Story é a **menor unidade de valor entregável** de um produto, escrita do ponto de vista de quem usa — não do ponto de vista técnico. Ela não descreve a solução: descreve a **necessidade** e o **motivo**. A solução nasce depois, a partir dela.

/ A HISTÓRIA DIZ

A necessidade

Quem precisa, **o quê** e **por quê**. É um pedaço de valor sob o olhar do usuário. A equipe ainda tem liberdade para propor a melhor solução.

/ A HISTÓRIA NÃO DIZ

A solução

Componente, tela, "botão azul". Isso vem depois — derivado dos critérios. Descrever a interface cedo demais **amarra o time** e mata alternativas melhores.

Regra de ouro: toda história começa numa dor real validada na pesquisa — nunca num palpite. Se você não consegue apontar o dado que sustenta a história, ela ainda não está pronta para o backlog.

Por que escrever assim

- Mantém o foco no **problema do usuário**, não na preferência de quem desenvolve
- Cria uma unidade pequena o bastante para caber numa sprint e ser priorizada
- Abre espaço para a equipe propor soluções — a história define o "o quê", não o "como"

● A FÓRMULA

Como • Quero • Para que.

Toda história segue exatamente este formato de três partes. Cada parte responde uma pergunta – e pular qualquer uma delas quebra a priorização.

Como [persona / tipo de usuário específico]
Quero [ação ou capacidade]
P/ [benefício • motivo • resultado esperado]
que

Cada parte tem um erro a evitar

/ COMO...

Quem?

Use persona **específica**.
Troque "usuário" por "usuário recém-cadastrado" ou "pai próximo à van".

/ QUERO...

O quê?

Descreva a **capacidade** ("registrar o preço"), não a interface ("um botão azul").

/ PARA QUE...

Por quê?

Nunca pule o benefício. Sem ele, o time não entende a **prioridade** nem propõe soluções melhores.

/ EXEMPLO CORRETO

Como **novo usuário**, quero **criar minha conta usando apenas e-mail e senha**, sem fornecer dados sensíveis no primeiro contato, para **manter o anonimato como diferencial do produto**.

/ EXEMPLO ERRADO • DESCREVE SOLUÇÃO

Quero uma tela de cadastro com campo de e-mail, campo de senha e um botão verde.

● O OBJETO DO BACKLOG

0 card completo.

Cada história vira um **card** com estes campos. É o objeto que você importa no Jira, Linear, Trello ou Notion — pronto para o board.

/ ID

Identificador único

Com prefixo do projeto. **ZEI-001**, **TE-014**, **EC-003**.

/ TÍTULO

Nome curto

Descritivo da funcionalidade. "Criação de conta com e-mail".

/ HISTÓRIA

Como · Quero · Para que

O texto no formato de três partes do capítulo anterior.

/ CRITÉRIOS

Aceitação

O que precisa ser verdade para a história estar "pronta". Apontam para a UI.

/ PRIORIDADE

Alta · Média · Baixa

A urgência relativa da história no backlog.

/ STORY POINTS

Esforço relativo

Fibonacci: **1 · 2 · 3 · 5 · 8 · 13**. Não é hora, é tamanho.

/ ÉPICO

Agrupamento temático

A que conjunto maior a história pertence.
Ex.: E1 — Onboarding.

/ SPRINT

Ciclo designado

O ciclo de desenvolvimento em que a história será entregue.

● O CORAÇÃO DO CARD

Critérios de aceitação.

É aqui que a história deixa de ser abstrata. Cada critério é uma **condição verificável**. Escreva de **5 a 7 por história** e faça cada um apontar para um componente ou comportamento de UI — porque é dali que você deriva as telas.

Exemplo · localização em tempo real

- Mapa exibe a posição atual da van atualizada a cada X segundos
- Pino do veículo com ícone distinto do pino de destino
- Badge de ETA (tempo estimado de chegada) visível sem rolar a tela
- Estado de "sinal perdido" tratado com mensagem clara
- Botão de centralizar o mapa na van

Cada critério já sugere um componente molecular

- posição no mapa · **MapPin**
- tempo de chegada · **EtaBadge**
- sinal perdido · **StatusCard**

É essa ponte — critério → componente — que faz o **System UX** funcionar. A história revela a necessidade; o critério revela a interface.

● AGRUPANDO AS HISTÓRIAS

Épicos.

Um **épico** é um conjunto de histórias que servem a um mesmo objetivo maior. Você organiza o backlog inteiro em épicos e dá a cada um uma **cor** — que vira a borda do card e facilita a leitura do board.

Exemplo • app de transporte escolar

E1	Onboarding & Conexão	· Roxo	· 5 histórias	· 26 pts
E2	Localização em Tempo Real	· Azul	· 5 histórias	· 29 pts
E3	Eventos do Trajeto	· Laranja	· 5 histórias	· 25 pts
E4	Segurança & Emergência	· Vermelho	· 3 histórias	
E5	Histórico & Transparência	· Verde	· 2 histórias	

5

épicos cobrem o produto inteiro neste exemplo

20

histórias distribuídas entre os épicos

1

cor por épico — leitura visual de relance

*A cor do épico **não é decoração**: ela ajuda o time a enxergar concentração de esforço e dependências de relance.*

● SEQUENCIANDO A ENTREGA

Sprints.

As histórias, agrupadas em épicos, são distribuídas em **sprints** (ciclos de 2 semanas). Ordene sempre dos **fundamentos para o avançado**: nada de gamificação no Sprint 1 se o login ainda não existe.

Sequência típica de um app

- 01 **Sprint 1 — Fundamentos:** cadastro, login, onboarding, navegação, home
- 02 **Sprint 2 — Core:** a feature que justifica o app existir
- 03 **Intermediários:** busca, comparação, listas, alertas
- 04 **Engajamento:** gamificação, social, rede
- 05 **Finais:** monetização, perfil/config, lançamento/viralização

/ COMO PROJETAR O PRAZO

Cada sprint tem um **tema** e um total de story points. Somando os pontos e dividindo pela **velocidade do time**, você projeta as semanas de desenvolvimento. É daqui que sai a estimativa de prazo — não de um chute.

*A ordem importa: **fundamento primeiro**. Toda feature avançada depende de uma base que precisa existir antes.*

● A ORDEM IMPORTA

O fluxo System UX.

Componente e tela vêm **depois** da história, nunca antes. Este é o caminho completo — da dor de pesquisa ao primeiro traço no Figma.

- 01 Discovery · pesquisa com usuários · identifica dores reais
- 02 Priorização · escolhe o problema mais latente (com dados)
- 03 User Stories · escreve Como / Quero / Para que
- 04 Critérios · condições que apontam para UI
- 05 Épicos + Sprints · agrupa e sequencia
- 06 Componentes · deriva os moleculares dos critérios
- 07 Telas · compõe as telas a partir dos componentes
- 08 Figma · só agora abre o Figma para desenhar

Princípio System UX: pensar molecular/componente antes de abrir o Figma. A história define a necessidade, os critérios revelam os componentes, os componentes montam as telas. O Figma é a última etapa, não a primeira.

● ENTREGÁVEL BÔNUS

Mapa de componentes & telas.

Ao final de cada backlog, vale entregar um **mapa** que faz a ponte história → interface. É o que você leva para o caderno de wireframes **antes** de abrir o Figma.

/ LISTA DE TELAS

Derivadas do backlog

Onboarding · Home · Mapa · Histórico... As telas que emergem do conjunto de histórias e critérios.

/ COMPONENTES MOLECULARES

Os que se repetem

StatusCard · MapPin · EtaBadge · TimelineEvent · DriverCard. Os blocos que aparecem em mais de uma tela.

Por que o mapa importa

- Revela os componentes que se **repetem** – base de um design system enxuto
- Mostra a cobertura: cada tela tem histórias por trás? cada história aparece em alguma tela?
- Dá ao time um vocabulário comum entre **produto, UX e dev** antes do desenho

*O mapa é a prova de que você pensou em **componente antes de pixel**. O wireframe vira montagem, não invenção.*

● ANTES DO BACKLOG

Checklist de qualidade.

Antes de mandar uma história para o backlog, confirme cada ponto. Se algum falhar, a história ainda **não está pronta**.

- A história nasce de uma dor **validada na pesquisa** (tem dado por trás)
- Está no formato **Como / Quero / Para que** completo – as três partes
- O "Como" usa uma persona **específica**, não "usuário" genérico
- O "Quero" descreve **capacidade**, não interface
- O "Para que" deixa o **benefício** claro
- Tem **5 a 7 critérios de aceitação** verificáveis
- Os critérios **apontam para componentes/comportamentos de UI**
- Tem ID, prioridade, story points, épico e sprint definidos
- Pertence a um **épico colorido** e a um **sprint sequenciado**

/ O TESTE FINAL

Consegue apontar **o dado de pesquisa** que justifica a história e **o componente** que cada critério sugere? Se sim, está pronta. Se não, volte ao discovery.

● COPIE E PREENCHA

Modelo pronto.

O template do card e um exemplo preenchido. Use no Claude Code como referência para gerar backlogs completos a partir de dados de pesquisa.

ID	[PREFIXO] - [NNN]
Título	[nome curto da funcionalidade]
Épico	[EX – Nome] · Cor: [cor]
Meta	Prioridade · Story Points · Sprint
História	Como [persona], quero [ação], para que [benefício]
Critér.	5 a 7 itens → cada um aponta para um componente

ZEI-001 · Criação de conta com e-mail

/ EXEMPLO

Épico: E1 – Onboarding & Conexão · Roxo · **Prioridade:** Alta · **8 pts** · Sprint 1

História: Como novo usuário, quero criar minha conta usando apenas e-mail e senha, para manter o anonimato como diferencial do produto.

Critérios de aceitação · ZEI-001

- Validação de formato de e-mail em tempo real
- Senha com regras de força visíveis — **PasswordStrengthMeter**
- Termos e política LGPD aceitos explicitamente — **ConsentCheckbox**
- Envio de e-mail de confirmação
- Estado de erro tratado com mensagem clara — **InlineError**

● O QUE FAZER AGORA

Transforme pesquisa em backlog pronto.

O System UX leva sua descoberta da dor à interface sem pular etapas: história, critério, componente, tela. O Figma vira a última etapa — e o backlog chega no board pronto para priorizar e estimar.

fihan.com.br